



CDA 650 — Special Topics
Experiential Projects in Artificial Intelligence and Data Science
Spring 2026

Vibration-Based Fault Analysis and Severity Estimation in Rotating Machinery

Project Book

Authors — Team Delta:

Nuthan Teja Reddy Endurthi
Venkata Sai Akhil Anga
Sai Krishna Anagani

Advisors:

Prof. Saverio Marrazzo
Prof. Adithya Narayanan
Mr. John Biondolillo

Spring 2026

Abstract

This project focuses on developing a data-driven approach to analyze vibration signals from rotating machinery and identify mechanical faults. The primary goal is not only to detect faults but also to move toward estimating their severity. Using triaxial vibration data collected from a controlled test rig, we explore three modeling approaches—feature-based machine learning, 1D CNN, and vibration-imaging techniques—to characterize fault conditions such as normal operation, unbalance, misalignment, bearing faults, and mechanical looseness.

In the initial phase, we focused on understanding the machinery setup, collecting vibration data under different conditions, and constructing a structured dataset. A feature engineering pipeline was implemented to extract meaningful time-domain and frequency-domain characteristics from the raw signals. A baseline machine learning model was developed to classify fault conditions, providing a strong foundation for subsequent work on severity estimation.

In the later phases, severity-based datasets were constructed for multiple fault conditions including unbalance, misalignment, and mechanical looseness. Multiple machine learning, deep learning, and time-series classification approaches were explored: Random Forest, ExtraTrees, XGBoost, LightGBM, 1D CNN, ResNet-based architectures, InceptionTime-inspired models, and MiniRocket-based approaches. Several experimental modifications to the rig were also performed, including structural reinforcement with additional bolts, rotor and bearing re-alignment, and sleeve-bearing replacement, to improve signal quality and reduce mechanical noise.

Comparative analysis demonstrated that feature-engineering-based methods provided strong interpretability and reliable classification performance, while MiniRocket-based models showed strong capability in capturing temporal vibration behavior directly from raw time-series signals with high computational efficiency. The project ultimately evolved into an integrated fault and severity monitoring framework combined with dashboard-oriented visualization for real-time machinery health monitoring and predictive maintenance.

Acknowledgements

We would like to express our sincere gratitude to Professor Saverio Marrazzo and Professor Adithya Narayanan for their invaluable guidance and support throughout this project. As the instructors of the course, they played a central role in coordinating with Machinery Monitoring Systems, LLC to identify a suitable data-driven problem and provided continuous academic and technical direction. We are especially grateful to Professor Marrazzo for his assistance in setting up the machinery and demonstrating methods for simulating mechanical faults, which formed the foundation of our data collection process.

We also extend our appreciation to Mr. John Biondolillo, Chief Technology Officer at Machinery Monitoring Systems, LLC, for providing access to high-quality wireless triaxial vibration sensors used in this study. His support ensured that our experiments were conducted with state-of-the-art hardware. Additionally, we thank Mr. Jesus Velasquez for his technical assistance.

Finally, we would like to acknowledge Sushruth for sharing insights from his prior work on a similar project. His initial guidance helped us better understand the problem context and scope, enabling us to begin our work with clarity and confidence.

Contents

List of Figures	vi
List of Tables	vii
1 Introduction	1
1.1 Background and Motivation	1
1.2 Problem Statement	1
1.3 Objectives	1
1.4 Scope of the Project	2
2 Experimental Setup and Rig Optimization	3
2.1 Rotating Machinery Test Rig	3
2.2 Data Acquisition and Initial Benchmarking	3
2.3 Fault Simulation and Scope Adjustments	3
2.3.1 Unbalance Condition Severity	4
2.3.2 Misalignment Condition Severity	4
2.3.3 Mechanical Looseness Condition Severity	4
2.4 Setup Optimization Experiments	4
2.4.1 Structural Reinforcement and Speed Calibration	4
2.4.2 Mechanical Re-alignment	5
2.4.3 Sleeve Bearing Replacement	5
2.5 Final Configuration for Severity Modeling	5
3 Dataset Preparation and Feature Engineering	6
3.1 Data Ingestion and Synchronization Logic	6
3.2 Signal Preprocessing and Normalization	6
3.3 Multi-Path Feature Generation	6
3.3.1 Statistical and Spectral Feature Engineering (ML Path)	6
3.3.2 Raw Signal Formatting (DL and MiniRocket Path)	7
3.4 Dataset Labeling and Severity Distribution	7
4 Modeling Architectures	8
4.1 Phase 1: Fault Prediction Architectures for Rig Benchmarking	8
4.1.1 Random Forest and Interpretable Baselines	8
4.1.2 Tuned 1D CNN for FFT Data	8
4.2 Phase 2: Severity Estimation Architectures	8
4.2.1 Feature-Based Machine Learning (Baselines)	8
4.2.2 Deep Learning Explorations	9
4.2.3 Selected Deployment Model: MiniRocket and Ridge Classifier	9
4.3 Rationale for Final Model Selection	9

5	Training Setup and Evaluation Metrics	10
5.1	Performance Evaluation Metrics	10
5.2	Stage 1 Training Setup: Rig Benchmarking	10
5.3	Stage 2 Training Setup: Severity Estimation	10
5.3.1	Hardware Environment	11
5.3.2	MiniRocket and Ridge Classifier Pipeline	11
5.3.3	Deep Learning and ML Baselines	11
6	Results and Discussion	12
6.1	Stage 1: Rig Optimization and Benchmarking Results	12
6.1.1	Baseline Setup and Initial Confusion	12
6.1.2	Impact of Physical Modifications	13
6.2	Stage 2: Severity Estimation Results	13
6.2.1	MiniRocket Performance Analysis	13
6.2.2	Speed Independence and Robustness	15
6.3	Comparative Model Analysis	15
6.4	Discussion	15
7	System Deployment	17
7.1	System Architecture and Tech Stack	17
7.2	Project Organization	17
7.3	Operating Modes	18
7.4	Backend Inference Logic	18
7.4.1	REST Batch Endpoint	19
7.4.2	WebSocket Stream Endpoint	19
7.4.3	Data Validation and Shape Enforcement	19
7.4.4	Local Normalization Protocol	19
7.4.5	Triplet Buffering for Streaming	19
7.4.6	ISO 20816-3 Severity Mapping	20
7.5	Pluggable Data Sources	20
7.6	Custom HTML Industrial Interface	21
7.6.1	Design System and Glanceable Awareness	21
7.6.2	Severity Timeline and Recent Predictions	21
7.6.3	Plot Panels	21
7.7	Local Deployment Workflow and Configuration	22
7.8	Extensibility Points	22
8	Conclusion and Future Work	23
8.1	Summary of Contributions	23
8.2	Experimental and Practical Limitations	23
8.2.1	Supply Chain and Logistics	23
8.2.2	Interpretability of the Selected Model	24
8.3	Future Work	24

8.3.1	Progressive Severity and Trajectory Modeling	24
8.3.2	Broadening the Modeling Search	24
8.4	Concluding Remarks	24
Bibliography		25

List of Figures

6.1	Confusion matrix for the initial (unoptimised) rig setup, showing the pronounced <i>Normal/Unbalance</i> overlap that motivated the physical optimisation experiments.	12
6.2	Progressive improvement of the rig benchmark: confusion matrices for setup versions v5 (bolting), v6 (rotor/bearing re-alignment), and v7 (sleeve-bearing replacement).	13
6.3	Final MiniRocket confusion matrices for the severity task — machine state (left) and severity bucket (right).	15

List of Tables

2.1	Physical configurations used to simulate each fault severity level.	4
3.1	Engineered features extracted per axis for the ML pipeline.	7
5.1	Stage 1 training configuration for rig benchmarking.	10
6.1	Confusion matrix of the final MiniRocket model — machine state head. Rows: actual state. Columns: predicted state.	14
6.2	Confusion matrix of the final MiniRocket model — severity bucket head. Rows: actual severity. Columns: predicted severity.	14
6.3	Final classification report of the deployment MiniRocket model. Head 1 reports machine-state classification; Head 2 reports ordinal severity classification.	14
6.4	Comparison of modeling paradigms on the severity task.	15
7.1	Deployment tech stack for <code>pmos</code> (Maintenance_OS).	17
7.2	Top-level project layout.	18
7.3	Operating modes exposed by the Maintenance_OS deployment.	18
7.4	Mapping of model severity classes to ISO 20816-3 Action Zones.	20
7.5	The three shipped data-source ingestors. Selected via <code>PMOS_INGESTOR</code> ; all are interchangeable behind the <code>BaseIngestor</code> ABC.	20
7.6	Diagnostic plot panels rendered in the Real-time and Batch views.	21

1. Introduction

1.1 Background and Motivation

Rotating machinery plays a critical role in modern industrial systems, and unexpected failures can lead to significant operational costs and unplanned downtime. Components such as motors, pumps, and gearboxes often operate under extreme mechanical stress, making their reliability a top priority across manufacturing and energy sectors.

The economic consequences of machine failure are substantial. Industry studies report that unplanned downtime costs manufacturers tens of billions of dollars annually, with some estimates placing the total cost to large industrial organizations at over \$1 trillion per year when accounting for lost production and recovery expenses. Operational surveys indicate that unplanned downtime can cost between \$125,000 and \$260,000 per hour on average. In high-value production environments, such as automotive manufacturing, the cost of a single hour of downtime can even exceed \$2 million.

Traditional maintenance approaches are often reactive and do not provide early warning signals. As a result, industries are shifting toward predictive maintenance strategies. Vibration analysis is widely used for this purpose because it directly reflects internal mechanical behavior. Characteristic patterns in vibration signals can indicate faults such as imbalance, misalignment, bearing damage, and mechanical looseness. This project leverages vibration sensor data to identify these faults and, crucially, moves beyond simple detection to understand their specific severity.

1.2 Problem Statement

While detecting the presence of a fault is important, the primary challenge is to *quantify* the severity of those faults—for example, determining the exact intensity of an imbalance or misalignment. Vibration signals are inherently complex, noisy, and influenced by various operating conditions like speed and load. Therefore, the challenge is to extract meaningful information from these signals and build a model that can differentiate between various fault categories while accurately estimating their severity levels.

1.3 Objectives

The primary objective of this project is to develop a data-driven framework for identifying mechanical faults and estimating their severity. The specific objectives are:

- Construct severity-based datasets for multiple fault conditions.
- Compare multiple machine learning, deep learning, and time-series approaches.
- Evaluate the effect of machinery setup modifications on signal quality.
- Identify the most effective modeling approach for fault severity estimation.
- Design a dashboard-oriented visualization system for real-time machinery monitoring.

1.4 Scope of the Project

The current scope focuses on vibration-based analysis for a single rotating machinery setup under controlled laboratory conditions. The scope includes:

- Controlled fault simulation and triaxial vibration data acquisition.
- Construction of datasets categorized by severity (Low, Medium, and High).
- Feature engineering using time-domain and frequency-domain analysis.
- Comparative analysis of multiple machine learning and deep learning models.
- Specialized time-series modeling using MiniRocket-based approaches.
- Experimental setup optimization, including structural reinforcement and bearing replacement, for improved signal quality.
- Development of a custom HTML-based dashboard UI for machinery health visualization.

The project focuses on five machine conditions: normal operation, unbalance, misalignment, bearing fault, and mechanical looseness. It does not address remaining useful life prediction or compound faults where multiple conditions coexist simultaneously.

2. Experimental Setup and Rig Optimization

This chapter describes the experimental environment and the iterative process used to optimize the rotating machinery test rig. Unlike the previous cohort, our team conducted a series of physical refinements to the machinery setup before finalizing the data collection for severity estimation. The goal of this phase was to ensure the highest signal quality by identifying and eliminating mechanical noise and baseline imbalances that could lead to model confusion.

2.1 Rotating Machinery Test Rig

The experiments were conducted using a laboratory-scale rotating machinery test rig designed to simulate common industrial faults. The system consists of a motor-driven shaft supported by bearing housings and a rotor assembly. A wireless triaxial vibration sensor provided by Machinery Monitoring Systems, LLC was mounted on the rig to capture vibration responses along three orthogonal axes: X , Y , and Z . This triaxial measurement provides a comprehensive representation of machine dynamics, allowing the system to capture complex fault signatures.

2.2 Data Acquisition and Initial Benchmarking

Data were initially collected at a rotational speed of 2000 RPM. Each recording session captured high-resolution time-series signals, which were subsequently processed into Fast Fourier Transform (FFT) representations. To evaluate the quality of the physical setup, we employed a 1D Convolutional Neural Network (1D CNN) tuned to FFT data as a diagnostic benchmark.

During initial testing, the benchmark model revealed a significant physical limitation in the rig setup. While the model could accurately identify bearing faults, mechanical looseness, and misalignment, it was almost entirely unable to distinguish the *Normal* state from the *Unbalance* state. Specifically, the majority of *Normal* samples were misclassified as *Unbalance*, suggesting that the rig possessed a baseline mechanical imbalance even when no fault was intentionally induced.

2.3 Fault Simulation and Scope Adjustments

The experimental phase was divided into two distinct modeling stages: Fault Prediction and Severity Estimation. For the initial Fault Prediction stage, five machine conditions were successfully simulated: Normal, Unbalance, Misalignment, Mechanical Looseness, and Bearing Fault.

However, a significant **scope adjustment** was made prior to the severity modeling phase. The project originally intended to model severity levels for bearing faults. Due to supply-chain issues that prevented the acquisition of the additional bearings required for varied fault testing, the severity estimation for bearing faults was eliminated from the final scope. Consequently, the severity modeling focused on the three remaining fault categories, simulated across three

rotational speeds (2200, 2700, and 3200 RPM) to ensure the models remained robust across different operating velocities.

The severity levels for each fault category are summarized in Table 2.1.

Table 2.1. Physical configurations used to simulate each fault severity level.

Fault Type	Severity	Physical Configuration
Unbalance	Low	Two small screws attached to the rotor.
	Medium	Four small screws attached to the rotor.
	High	Four small screws + two mid-size screws (maximum centrifugal force).
Misalignment	Low	Two thick washers inserted under the bearing housing.
	Medium	Two thick washers + two thin washers.
	High	Two thick washers + three thin washers (extreme offset).
Mechanical Looseness	Low	Mounting screws loosened by 180°.
	Medium	Mounting screws loosened by 270°.
	High	Mounting screws loosened by 360° (highly unstable profile).

2.3.1 Unbalance Condition Severity

Unbalance was simulated by attaching discrete weights to the rotor to disturb the mass distribution. Combinations of screw sizes were used to achieve precise increments in centrifugal force, as listed in Table 2.1.

2.3.2 Misalignment Condition Severity

Misalignment was introduced by placing washers beneath the bearing housing to create a vertical offset, altering the shaft alignment relative to the motor.

2.3.3 Mechanical Looseness Condition Severity

Mechanical looseness was simulated by loosening the mounting screws that secure the bearing housing to the rig base. The intensity of looseness was quantified by the degree of rotation applied to the screws.

2.4 Setup Optimization Experiments

To resolve the baseline imbalance and noise issues identified during benchmarking, a series of seven iterative physical modifications (v1 through v7) were performed.

2.4.1 Structural Reinforcement and Speed Calibration

Initial experiments were conducted at 2000 RPM. We hypothesized that resonance and vibrations from the rig base were interfering with the sensor data. Structural reinforcement was performed by bolting extra screws to the base of the rig. This modification significantly enhanced stability, allowing us to increase the rotational speed to a stable 2500 RPM for all subsequent experiments.

2.4.2 Mechanical Re-alignment

We suspected the baseline imbalance was caused by asymmetric positioning of the rotor and bearings. In version 5 of the setup, the center bearing was moved closer to the motor and the rotor was centered on the shaft. This re-alignment provided the first successful differentiation between the *Normal* and *Unbalance* states in the benchmark confusion matrix.

2.4.3 Sleeve Bearing Replacement

To further reduce signal interference, the ball bearing located near the motor was replaced with a sleeve bearing. This change was motivated by audible noise from the original roller bearing, which we suspected was contaminating the vibration signatures of other fault states. The use of a sleeve bearing effectively eliminated these unwanted noise artifacts.

2.5 Final Configuration for Severity Modeling

The final optimized configuration used for all severity-based data collection was an *equi-distant* arrangement: the motor, the sleeve bearing, and the rotor were positioned at equal intervals along the shaft to maximize stability and signal clarity.

By comparing the benchmark confusion matrices across these experiments, it was evident that the equi-distant sleeve-bearing setup provided the most distinct separation between fault classes. This optimized physical environment allowed us to proceed with confidence to the collection of the high-fidelity data required for multi-level severity estimation.

3. Dataset Preparation and Feature Engineering

This chapter describes the transformation of raw sensor signals into structured digital representations suitable for fault and severity analysis. The objective was to maintain high data integrity through strict synchronization protocols and to prepare multiple feature sets to support the evaluation of both traditional machine learning and advanced time-series architectures.

3.1 Data Ingestion and Synchronization Logic

The digital pipeline began with the acquisition of raw time-series signals from the wireless triaxial sensor. The sensor streamed individual waveform frames for the X , Y , and Z axes asynchronously. A critical challenge in this phase was ensuring that the data used for modeling represented physically consistent snapshots of the machinery’s state.

In the initial raw capture, the dataset consisted of approximately 73,000 individual sensor frames. To ensure data integrity, we enforced a strict synchronization protocol: a triaxial sample was only recorded if X , Y , and Z frames were received in a perfectly uninterrupted temporal sequence. Frames that arrived out of order or lacked matching axes were discarded. This rigorous filtering process reduced the dataset to approximately 21,304 high-quality synchronized samples, each represented as a 1024×3 array.

3.2 Signal Preprocessing and Normalization

Following synchronization, each triaxial window underwent signal conditioning to prepare it for modeling. The primary preprocessing step was direct current (DC) removal, achieved by subtracting the mean value from each 1024-point window. This operation eliminated constant offsets introduced by the sensor electronics while preserving the dynamic vibrational characteristics essential for fault diagnosis.

To prevent any single vibration axis from dominating the learning process due to differences in physical magnitude, we applied axis-wise standardization. The mean and standard deviation were computed independently for the X , Y , and Z channels, and the signals were scaled accordingly. This step ensured that the models could learn the unique contributions of each axis equally during the fault and severity classification tasks.

3.3 Multi-Path Feature Generation

To facilitate the comparison of diverse modeling paradigms, we constructed two distinct data pathways from the synchronized samples.

3.3.1 Statistical and Spectral Feature Engineering (ML Path)

For the feature-based models such as XGBoost and LightGBM, a manual extraction process was implemented to translate raw waveforms into a structured attribute matrix. The metrics computed for each axis are summarized in Table 3.1.

Table 3.1. Engineered features extracted per axis for the ML pipeline.

Category	Features
Time-Domain	Root Mean Square (RMS), standard deviation, peak-to-peak amplitude, maximum/minimum, kurtosis, variance.
Frequency-Domain	Dominant frequency components, spectral energy (via FFT).
Cross-Axis	Mean RMS across all axes and other holistic aggregates.

3.3.2 Raw Signal Formatting (DL and MiniRocket Path)

For the final deployment model (MiniRocket) and the deep learning baselines (CNN and ResNet), the synchronized 1024×3 segments were used directly. These windows were generated using a non-overlapping sequential cut. This “straight sequential” approach ensured that each sample represented a unique snapshot of the machine’s behavior without the potential temporal bias introduced by sliding-window overlaps.

3.4 Dataset Labeling and Severity Distribution

The dataset was labeled using a multi-class schema that integrated fault type, sensor position, operating speed, and physical intensity. To maintain consistency across thousands of files and various experimental iterations, we utilized a structured naming convention:

`FaultName_Position_Duration_RPM_Severity_Extra.jsonl`

For example:

`MechanicalLooseness_Shaftend_10mins_2200rpm_high_360deg.jsonl`

The modeling objective was to classify samples into ten distinct states: *Normal*, *Unbalance* (Low, Medium, High), *Misalignment* (Low, Medium, High), and *Mechanical Looseness* (Low, Medium, High). The severity-based dataset was designed to be balanced across the three fault intensities so that the models did not develop a bias toward any specific level of damage. The *Normal* class contained fewer samples than the aggregated fault classes; because healthy operation does not possess severity gradations, the *Normal* state was treated as a single baseline class. This distribution accurately reflects industrial maintenance scenarios where the healthy state is a fixed reference point against which various levels of mechanical degradation are measured.

4. Modeling Architectures

This chapter outlines the development and evaluation of the computational models used for both fault prediction and severity estimation. The modeling process was conducted in two distinct phases: an initial benchmarking phase to validate the experimental setup, and a secondary phase focused on multi-level severity classification. While several architectures were explored, the final system utilizes a MiniRocket-based pipeline for deployment.

4.1 Phase 1: Fault Prediction Architectures for Rig Benchmarking

The first phase of modeling focused on identifying the five core machine conditions to verify the stability of the experimental setup. During this stage, we utilized two primary architectures as diagnostic tools to “benchmark” the physical rig.

4.1.1 Random Forest and Interpretable Baselines

A Random Forest model was initially implemented using the manual features described in Chapter 3. This model provided a baseline for class separability and served as an early indicator of mechanical-noise issues in the initial rig setup.

4.1.2 Tuned 1D CNN for FFT Data

To achieve higher sensitivity during benchmarking, we developed a 1D Convolutional Neural Network (1D CNN) specifically tuned to process FFT-transformed vibration data. This model outperformed the Random Forest baseline by providing near-perfect separation for most fault classes, only struggling with the overlap between *Normal* and *Unbalance* states. For ease of comparison across the seven rig-optimization experiments (v1–v7), the output of this 1D CNN was used as the standard metric for rig quality.

4.2 Phase 2: Severity Estimation Architectures

Once the rig was optimized, the modeling objective transitioned to the more complex task of differentiating between *Low*, *Medium*, and *High* severity levels for each fault.

4.2.1 Feature-Based Machine Learning (Baselines)

We evaluated several gradient-boosted decision tree architectures using the statistical and spectral features extracted by our team. Models such as XGBoost, LightGBM, and ExtraTrees were tested because of their strong interpretability in industrial contexts. Among these, XGBoost and LightGBM demonstrated consistent performance with accuracies around 92%. These models were particularly useful for identifying which specific time-domain or frequency-domain features were most indicative of increasing fault intensity.

4.2.2 Deep Learning Explorations

To reduce the need for manual feature engineering, we explored end-to-end deep learning models including 1D CNNs, ResNet, and InceptionTime-inspired architectures. The initial plan was to utilize a CNN-based architecture for deployment because of its fast inference times. However, during training for the severity task, these models struggled to consistently differentiate between overlapping severity levels under noisy conditions. As a result, we moved toward more specialized time-series classification methods.

4.2.3 Selected Deployment Model: MiniRocket and Ridge Classifier

The final model selected for the monitoring framework is based on the MiniRocket (RandOm Convolutional Kernel Transform) architecture combined with a Ridge Classifier. Unlike traditional CNNs that learn filters through gradient descent, MiniRocket applies a fixed set of 10,000 random convolutional kernels to the raw triaxial signals. This approach effectively transforms the time-series data into a high-dimensional feature space that captures subtle temporal patterns missed by other models.

4.3 Rationale for Final Model Selection

The decision to fix on MiniRocket for the final deployment was based on a comprehensive evaluation of performance metrics beyond simple accuracy. While deep learning models were the initial target, MiniRocket provided a superior balance of training stability and predictive robustness on the available dataset.

- **Performance Metrics:** The MiniRocket model achieved near-perfect scores across all severity classes, with Precision, Recall, and F1-scores of approximately 1.00 for every category—including the difficult-to-distinguish mechanical-looseness severity levels.
- **Class Separation:** The final confusion matrix demonstrates that the MiniRocket architecture, supported by the optimized equi-distant sleeve-bearing setup, successfully eliminated the misclassifications observed in earlier modeling attempts.
- **Efficiency:** By using the default kernel configuration, the model remained computationally efficient enough to support real-time inference within the custom HTML user interface, without requiring the extensive compute resources needed for deeper CNN architectures.

5. Training Setup and Evaluation Metrics

This chapter documents the training configurations, hardware environments, and performance measures used across both stages of the project. To ensure transparency and reproducibility, we define the specific optimization parameters for the rig-benchmarking models and the final severity estimation pipeline.

5.1 Performance Evaluation Metrics

We evaluated the performance of our models using a combination of aggregate and class-wise metrics. These metrics allowed us to assess not only the overall accuracy of the system, but also the reliability of individual fault and severity predictions.

- **Accuracy:** Overall proportion of correct predictions across all samples.
- **Precision:** Reliability of a positive prediction for a specific class (ability not to label a negative sample as positive).
- **Recall:** Ability of the model to find all positive samples for a specific class.
- **F1-Score:** Harmonic mean of precision and recall. This metric served as our primary indicator of success for severity estimation, as it balances detection sensitivity and prediction reliability.

5.2 Stage 1 Training Setup: Rig Benchmarking

For the initial stage of the project, we focused on training models to identify the five core machine states to validate the physical rig. The setup was designed for rapid iteration and diagnostic clarity, and is summarized in Table 5.1.

Table 5.1. Stage 1 training configuration for rig benchmarking.

Model	Configuration
Random Forest Classifier	300 estimators; fixed random seed = 42 for deterministic results. Operates on the manually engineered feature set.
1D Convolutional Neural Network	Adam optimizer, learning rate 0.001, batch size 32, sparse categorical cross-entropy loss. Trained on FFT-transformed data; used as the “gold standard” for evaluating rig optimizations (v1–v7).

5.3 Stage 2 Training Setup: Severity Estimation

The second stage of the project involved high-fidelity modeling of fault intensities. Due to the computational demands of processing the full severity-based dataset, we utilized high-performance hardware and specialized time-series pipelines.

5.3.1 Hardware Environment

All Stage 2 experiments were conducted using high-memory cloud environments equipped with **NVIDIA A100** GPU accelerators. These resources were essential for handling the large-scale kernel transformations required by the MiniRocket architecture and for maintaining stable training times across various exploratory deep learning models.

5.3.2 MiniRocket and Ridge Classifier Pipeline

The final deployment architecture used the MiniRocket multivariate transformation followed by a Ridge Classifier with cross-validation. To maintain consistency and efficiency under project time constraints, we used the default hyperparameter settings for the MiniRocket transform, applying 10,000 random convolutional kernels to the raw triaxial vibration windows.

The classification head was implemented using `RidgeClassifierCV`, which performs an efficient leave-one-out cross-validation to select the optimal regularization strength (α) from a predefined logarithmic scale. This approach ensured that the model remained robust to noise without requiring manual hyperparameter tuning.

5.3.3 Deep Learning and ML Baselines

Preliminary deep learning models—including standard 1D CNNs—were trained using the Adam optimizer with a learning rate of 0.0005 to encourage stable convergence on the complex ten-class severity task. Feature-based baselines, such as XGBoost, were trained using default parameters to provide a quick reference point for interpretability and feature-importance analysis.

6. Results and Discussion

This chapter presents the experimental results obtained throughout the project, following the two-stage progression from rig optimization to multi-level severity estimation. We first analyze how physical setup modifications directly resolved baseline signal interference, followed by a detailed evaluation of the final MiniRocket-based severity model.

6.1 Stage 1: Rig Optimization and Benchmarking Results

The primary objective of the first stage was to ensure the mechanical test rig provided a clean “Normal” baseline. Our initial results demonstrated that without structural reinforcement and precise alignment, modeling fault states is significantly hampered by mechanical noise.

6.1.1 Baseline Setup and Initial Confusion

In the initial configuration, the benchmark 1D CNN model demonstrated an inability to distinguish between the *Normal* state and the *Unbalance* state. As shown in Figure 6.1, nearly all *Normal* samples were misclassified as *Unbalance*. This confirmed our hypothesis that the rig was physically unbalanced in its “factory” state, producing vibrational signatures that masked the transition to a healthy baseline.

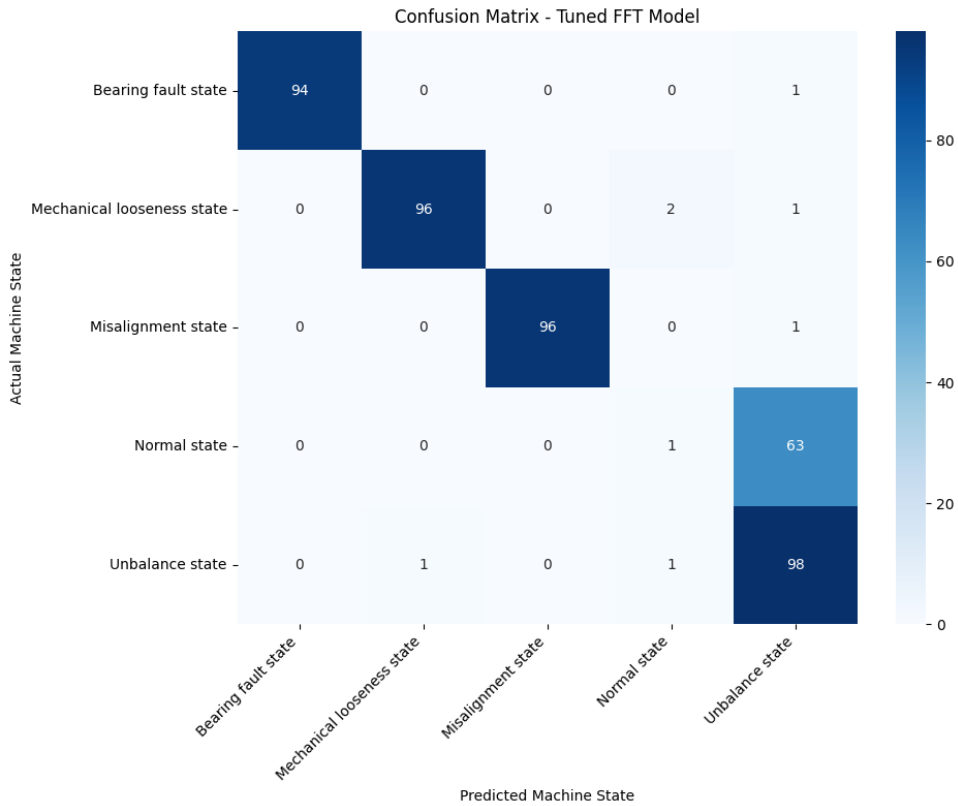


Figure 6.1. Confusion matrix for the initial (unoptimised) rig setup, showing the pronounced *Normal*/*Unbalance* overlap that motivated the physical optimisation experiments.

6.1.2 Impact of Physical Modifications

The iterative optimization experiments (v4 through v7) provided quantifiable improvements in class separability. The progression is visualised in Figure 6.2.

- **Bolting and Reinforcement:** Bolting the rig to the base reduced resonance and allowed for stable data collection at higher rotational speeds (up to 2500 RPM and eventually 3200 RPM).
- **Rotor and Bearing Re-alignment:** Re-aligning the center bearing and rotor was the critical turning point that allowed the model to finally differentiate between *Normal* and *Unbalance* states.
- **Sleeve Bearing Replacement:** Replacing the ball bearing near the motor eliminated high-frequency noise artifacts. As evidenced in the v6 and v7 confusion matrices, this change produced a nearly clean diagonal, representing a perfectly optimised physical baseline for severity modeling.

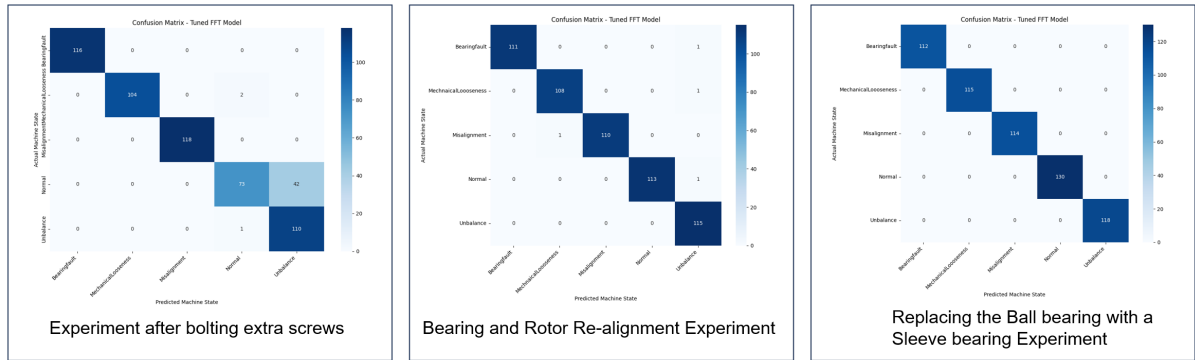


Figure 6.2. Progressive improvement of the rig benchmark: confusion matrices for setup versions v5 (bolting), v6 (rotor/bearing re-alignment), and v7 (sleeve-bearing replacement).

6.2 Stage 2: Severity Estimation Results

After fixing the physical setup, we applied the MiniRocket-based pipeline to the ten-class severity task across three rotational speeds (2200, 2700, and 3200 RPM).

6.2.1 MiniRocket Performance Analysis

The MiniRocket architecture achieved near-perfect classification performance across all ten classes. Unlike earlier deep-learning attempts, MiniRocket effectively disentangled the subtle differences between *Low*, *Medium*, and *High* severity levels for *Unbalance*, *Misalignment*, and *Mechanical Looseness*.

The final confusion matrices for the deployment MiniRocket model are reported as tables below; the original heat-map renderings are reproduced in Figure 6.3. Table 6.1 shows the machine-state head, and Table 6.2 shows the severity-bucket head.

Table 6.1. Confusion matrix of the final MiniRocket model — machine state head. Rows: actual state. Columns: predicted state.

Actual / Predicted	MechanicalLooseness	Misalignment	Normal	Unbalance
MechanicalLooseness	832	0	0	0
Misalignment	0	1030	0	0
Normal	0	0	325	4
Unbalance	2	0	4	1043

Table 6.2. Confusion matrix of the final MiniRocket model — severity bucket head. Rows: actual severity. Columns: predicted severity.

Actual / Predicted	Baseline	High	Low	Medium
Baseline	326	0	2	1
High	1	857	4	4
Low	1	8	992	9
Medium	0	3	4	1028

Table 6.3. Final classification report of the deployment MiniRocket model. Head 1 reports machine-state classification; Head 2 reports ordinal severity classification.

Class	Precision	Recall	F1-score	Support
<i>Head 1: Machine State Classification (MiniRocket)</i>				
MechanicalLooseness	1.00	1.00	1.00	832
Misalignment	1.00	1.00	1.00	1030
Normal	0.99	0.99	0.99	329
Unbalance	1.00	0.99	1.00	1049
Accuracy			1.00	3240
Macro avg	1.00	1.00	1.00	3240
Weighted avg	1.00	1.00	1.00	3240
<i>Head 2: Ordinal Severity Classification (MiniRocket)</i>				
Baseline	0.99	0.99	0.99	329
High	0.99	0.99	0.99	866
Low	0.99	0.98	0.99	1010
Medium	0.99	0.99	0.99	1035
Accuracy			0.99	3240
Macro avg	0.99	0.99	0.99	3240
Weighted avg	0.99	0.99	0.99	3240

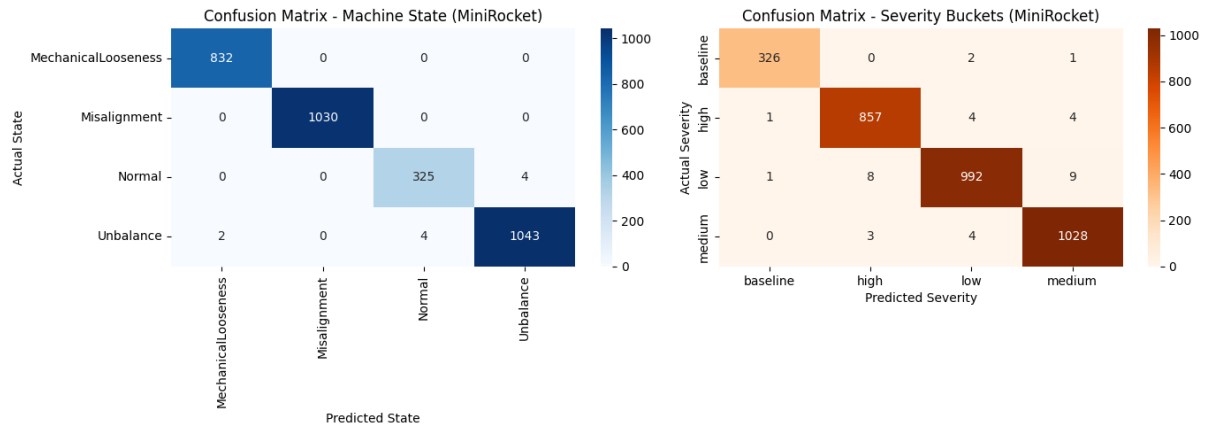


Figure 6.3. Final MiniRocket confusion matrices for the severity task — machine state (left) and severity bucket (right).

6.2.2 Speed Independence and Robustness

A significant finding of this work is the model’s ability to maintain high accuracy regardless of the motor’s rotational speed. By training on data from 2200, 2700, and 3200 RPM, the system learned to prioritize the fault signatures over the fundamental frequency of the machine. This ensures that the severity estimation remains accurate during real-world speed fluctuations.

6.3 Comparative Model Analysis

During the development phase, we compared multiple modeling paradigms. The results summarized in Table 6.4 highlight the superiority of the time-series kernel approach for this specific dataset.

Table 6.4. Comparison of modeling paradigms on the severity task.

Category	Key Model	Accuracy	Observations
Traditional ML	Random Forest	~ 88%	Good baseline but limited temporal capture.
Feature-Based	XGBoost / LightGBM	~ 92%	Strongest feature-engineered performers.
Deep Learning	1D CNN	Moderate	Required more data for stable convergence.
Time-Series (kernel)	MiniRocket	~ 99.9%	Highest performance and efficient temporal learning.

6.4 Discussion

The success of MiniRocket over deeper CNN architectures can be attributed to its “Random Convolutional Kernel Transform.” While deep learning models like the 1D CNN require large amounts of data to learn effective filters via gradient descent, MiniRocket utilizes a vast set of fixed, random kernels to extract complex temporal features immediately. Given our synchronized dataset of roughly 21,000 samples, MiniRocket provided a more stable and high-performing solution than the planned deep learning architectures.

Furthermore, the “failed” early experiments with the rig setup underscore the importance of mechanical engineering in data-driven diagnostics. No amount of modeling could resolve the initial *Normal/Unbalance* confusion until the rotor and bearings were physically re-aligned. The final system stands as a successful integration of high-quality physical data and advanced time-series modeling.

7. System Deployment

This chapter details the technical implementation of the vibration-based fault and severity monitoring system, internally codenamed *Maintenance_OS* (package name `pmos`). While the training phase utilized high-performance compute resources, the deployment is specifically optimized for local workstation execution. By transitioning from the previous cohort’s Streamlit-based approach to a modular FastAPI backend, a vanilla-JavaScript single-page frontend, and a pluggable data-source layer, the system provides a lightweight, industry-ready solution that can run on a laptop yet swap to a live serial sensor with a single environment variable.

A defining characteristic of the deployment is its commitment to *interchangeable components*. The model layer sits behind a `Predictor` abstract base class, and the streaming data layer sits behind a `BaseIngestor` abstract base class. Replacing the underlying MiniRocket model with a future CNN or Transformer, or replacing the synthetic data source with a live UART feed, requires no changes to the API surface, the frontend, or any client code. Severity outputs are mapped to **ISO 20816-3 Action Zones** (A, B, C, D), providing a standards-compliant interpretation alongside the raw model class label.

7.1 System Architecture and Tech Stack

The deployment is organized into a modular structure consisting of a dedicated backend inference service, a real-time streaming layer, and a responsive frontend dashboard. The system is built using Python 3.9+ and a specialized stack designed for time-series processing and low-latency streaming. The major components are summarized in Table 7.1.

Table 7.1. Deployment tech stack for `pmos` (*Maintenance_OS*).

Component	Role
FastAPI + Uvicorn	Backend API and WebSocket server; serves the static frontend from the same port. High performance with native asynchronous support.
Pydantic / pydantic-settings	Strict data validation of incoming triaxial vibration windows and env-driven configuration via <code>PMOS_*</code> variables.
Sktime + Scikit-Learn	MiniRocket multivariate transform and dual <code>RidgeClassifierCV</code> inference (machine state head + severity head).
Joblib	Efficient loading of the pre-trained model bundle (<code>minirocket_pipeline.joblib</code>).
NumPy / SciPy pyserial	Signal preprocessing, FFT, Hilbert envelope, and per-axis standardization. Live UART ingestion from the wireless triaxial sensor at 921 600 baud (the serial ingestor mode).
Vanilla JS + uPlot	Frontend SPA. ES modules with no build step; <code>uPlot</code> for high-performance, low-overhead chart rendering.
Custom CSS design system	Dark industrial control-room theme with severity-keyed colors and peripheral-vision state cues.

7.2 Project Organization

The repository is structured to ensure clarity, ease of maintenance, and a clean separation between the backend package, the frontend SPA, the shipped model artefacts, and the test suite.

The top-level layout is summarized in Table 7.2.

Table 7.2. Top-level project layout.

Folder	Purpose
<code>src/pmos/</code>	Backend Python package. Contains <code>core/</code> (Pydantic schemas, severity and ISO enums), <code>models/</code> (Predictor ABC, MiniRocketPredictor, registry), <code>inference/</code> (PredictionService facade, ISO 20816-3 mapping), <code>pipeline/</code> (CSV and JSONL loaders, DC-removal preprocessing, time/FFT/envelope plot generators), <code>streaming/</code> (triplet buffer, ingestor ABC and implementations, stream manager), and <code>api/</code> (FastAPI app factory and route modules for <code>/health</code> , <code>/api/meta</code> , <code>/api/batch/predict</code> , and <code>/ws/stream</code>).
<code>frontend/</code>	Static single-page application served by FastAPI's <code>StaticFiles</code> . Vanilla HTML with ES modules; three views (Batch / Real-time / Recent Predictions); uPlot wrappers for time-domain, FFT, and envelope-spectrum plots; severity timeline strip; stale-data detection.
<code>models/</code>	Serialized model bundle (<code>minirocket_pipeline.joblib</code>) and an accompanying <code>model_card.yaml</code> documenting the input contract, output classes, and the ISO 20816-3 mapping reference.
<code>tests/</code>	Split into <code>unit/</code> (predictor, loaders, triplet buffer, ingestors, ISO mapping) and <code>integration/</code> (REST batch endpoint and WebSocket stream roundtrips).
<code>scripts/</code>	Developer tooling: a PowerShell server runner and a standalone WebSocket replay client that prints prediction frames to the console.

7.3 Operating Modes

The deployment exposes three distinct operating modes to the operator, each backed by a different endpoint. They are summarized in Table 7.3.

Table 7.3. Operating modes exposed by the `Maintenance_OS` deployment.

Mode	Behaviour	Endpoint
CSV Batch Analysis	Upload a single time-window (CSV or JSONL); returns the predicted state, severity, ISO zone, and plot data (time-domain, FFT, envelope spectrum, per-axis statistics).	POST <code>/api/batch/predict</code>
Real-time Sensor Monitoring	WebSocket stream of prediction frames from the active ingestor (synthetic / replay / serial). Optional plot frames per window; per-client <i>eco mode</i> suppresses the heavier plot payload.	WS <code>/ws/stream</code>
Recent Predictions	Operator-facing log of recent predictions with state, severity, and dedup counts. Consecutive identical predictions collapse to a single row with an <code>×count</code> multiplier, so operators see at a glance how many sample windows agreed on a state.	Frontend page

7.4 Backend Inference Logic

The backend exposes two complementary inference paths: a synchronous REST endpoint for one-shot diagnostic uploads, and an asynchronous WebSocket endpoint for continuous monitoring.

Both paths share the same underlying `PredictionService`, which guarantees that batch and streaming predictions are produced by identical model code.

7.4.1 REST Batch Endpoint

The `POST /api/batch/predict` endpoint accepts `multipart/form-data` with a `file` field (CSV or JSONL) and an `rpm` form field. CSV uploads use a simple `axis,s0,s1,...,s1023` layout with one row per axis; JSONL uploads use the sensor-native format (one record per axis-window), and the backend selects the most recent valid $X/Y/Z$ triplet. The response combines a `Prediction` object (state, severity, per-head confidence, ISO zone) with a `PlotData` payload (time-domain samples, FFT magnitude, envelope spectrum, per-axis statistics).

7.4.2 WebSocket Stream Endpoint

The `WS /ws/stream` endpoint pushes server-to-client messages in three flavours: `status` (sent on connect, reporting the active ingestor, RPM, and eco-mode state), `prediction` (sent for every full triplet, carrying a `PredictionFrame`), and `plot` (sent alongside each prediction unless eco mode suppresses it). Clients may also send `set_eco` to toggle plot frames on a per-client basis, `set_rpm` to update the server-wide RPM live (since one machine has a single instantaneous speed), and `ping` to keep the connection alive.

7.4.3 Data Validation and Shape Enforcement

Upon receiving a request from either path, the backend validates that the input strictly follows the model contract: a $(N, 3, 1024)$ float tensor in axis order X, Y, Z , plus a scalar `rpm`. Each 1024-sample axis-window corresponds to 20.48 ms of vibration at the 50 kHz sampling rate. Pydantic schemas reject malformed sensor packets and out-of-order axis data before they reach the model, eliminating a class of silent errors that would otherwise be difficult to debug under live serial streaming.

7.4.4 Local Normalization Protocol

To maintain accuracy across deployment hardware, the backend replicates the training-time preprocessing exactly. Per-axis mean and standard-deviation statistics are bundled inside the `minirocket_pipeline.joblib` artefact and applied at inference time, ensuring the model receives signals in the expected distribution regardless of the host machine. The accompanying `model_card.yaml` documents the input contract, the class lists for both heads, and the ISO 20816-3 mapping that the deployment uses to translate severity classes into Action Zones.

7.4.5 Triplet Buffering for Streaming

The streaming layer relies on a small but critical component, the `TripletBuffer`, which holds the most recent window for each of the three axes and emits a fully formed $(3, 1024)$ triplet to the predictor only when all three axes are present and time-aligned within a configurable freshness window (`PMOS_TRIPLET_MAX_AGE_S`, default 1.0 second). Frames that arrive out of

order, are missing an axis, or fall outside the freshness window trigger a buffer reset rather than producing a misaligned prediction. This is the runtime counterpart to the strict synchronization protocol described in Chapter 3 for the training dataset.

7.4.6 ISO 20816-3 Severity Mapping

The raw severity output of the model (*baseline*, *low*, *medium*, *high*) is translated through a lookup defined in the model card into the corresponding ISO 20816-3 Action Zone, providing operators with a standards-compliant interpretation rather than only a model class label. The mapping used by the deployment is shown in Table 7.4.

Table 7.4. Mapping of model severity classes to ISO 20816-3 Action Zones.

Severity	ISO Zone	Operational Interpretation
Baseline	A	Good — newly commissioned condition.
Low	B	Acceptable — suitable for unrestricted long-term operation.
Medium	C	Unsatisfactory — not normally suitable for continuous operation.
High	D	Unacceptable — sufficient severity to cause damage.

7.5 Pluggable Data Sources

A key extensibility feature of the deployment is its pluggable ingestion layer. The active data source is selected at startup via the `PMOS_INGESTOR` environment variable, with no code changes required. Three implementations ship with the system, summarized in Table 7.5.

Table 7.5. The three shipped data-source ingestors. Selected via `PMOS_INGESTOR`; all are interchangeable behind the `BaseIngestor` ABC.

Ingestor	PMOS_INGESTOR	Description
Synthetic	<code>synthetic</code>	Generates sine + harmonics + noise per axis. Predictions on this signal are <i>not meaningful</i> ; the purpose is to verify the full pipeline end-to-end without hardware. This is the default, enabling clone-and-run demos.
Replay	<code>replay</code>	Plays back a recorded JSONL capture at original timing (<code>PMOS_REPLAY_REALTIME</code>) or as-fast-as-possible, optionally looping (<code>PMOS_REPLAY_LOOP</code>). Useful for demos with realistic data, regression checks, and validating the model on labelled files.
Serial	<code>serial</code>	Live UART ingestion via <code>pyserial</code> , parsing the ASCII-hex frame protocol emitted by the firmware capture script (<code>get_data_wave_uart.py</code>). Each line is <code><device_id> <axis> <hex_s0> ...<hex_s1023></code> ; DC removal is applied host-side before the window is buffered. Baud rate (<code>PMOS_SERIAL_BAUDRATE</code> , default 921 600) matches the firmware.

The serial ingestor also propagates the firmware-emitted `device_id` into every prediction frame, allowing the frontend to populate its asset card and “source” line directly from the hardware. If the firmware stops emitting frames, the dashboard’s stale-data detection fires after five seconds and the topbar transitions to an amber `SENSOR STALE` indicator; a full WebSocket disconnect produces a red `DISCONNECTED` state. This “fail-loud, never silent” policy ensures that an operator can never be misled by a frozen display.

7.6 Custom HTML Industrial Interface

A major differentiator of this work is the move to a bespoke frontend rather than a generic Python dashboarding tool such as Streamlit or Dash. The custom interface uses vanilla HTML and ES modules with `uPlot` for chart rendering, avoiding the heavy server-side session state and full-page re-run overhead of script-driven Python dashboards, and making it ideal for execution on standard workplace computers. There is no build step in development: `index.html` references `uPlot` from a CDN and loads ES modules directly.

7.6.1 Design System and Glanceable Awareness

The interface adopts a dark industrial control-room aesthetic with a deliberate *glanceable-awareness* pattern adapted from clinical patient monitors. The entire viewport carries a coloured border and inner glow keyed to the current severity (green for healthy, yellow for low, orange for medium, red for high), so an operator picks up machine state from peripheral vision without focusing on any single numeric value. The hero card reinforces this with a tinted background, the state name, a severity badge, the fault confidence in large monospace type, and the severity confidence as a small footnote.

7.6.2 Severity Timeline and Recent Predictions

A 60-column severity timeline strip beneath the hero on the real-time view encodes the model's recent decisions as colored bands: a solid band indicates a consistent prediction across many windows, while a speckled band indicates the model is flickering between severities — itself diagnostically informative. The Recent Predictions view provides the full historical log with time, state, severity, and an `×count` multiplier that collapses consecutive identical predictions into a single row.

7.6.3 Plot Panels

Each prediction is accompanied (unless suppressed by eco mode) by a four-panel diagnostic view. The panels are summarized in Table 7.6. The envelope-spectrum panel is included specifically because the Hilbert-envelope FFT is the canonical technique for surfacing bearing characteristic frequencies, complementing the bearing-fault classification work discussed earlier in this report.

Table 7.6. Diagnostic plot panels rendered in the Real-time and Batch views.

Panel	Source
Time-Domain Waveform	Raw 3-axis samples after DC removal.
FFT Magnitude Spectrum	<code>np.fft.rfft</code> per axis.
Envelope Spectrum	FFT of the Hilbert envelope per axis; surfaces bearing characteristic frequencies.
Diagnostic Statistics	Per-axis RMS, peak, crest factor, and kurtosis.

The *y*-axes use asymmetric exponential-moving-average smoothing (fast growth, slow decay) so that genuine spikes do not lock the scale and steady signals do not jitter.

7.7 Local Deployment Workflow and Configuration

The system is initiated locally by running `uvicorn pmos.api.app:app`, which starts the FastAPI server, serves the static frontend on the same port, and exposes the API and WebSocket endpoints. The operator opens the dashboard in a standard web browser; an auto-generated OpenAPI UI is available at `/docs` for inspecting and testing the REST surface. This local-first design ensures that machine data never leaves the facility, providing the security and low latency required for practical industrial predictive maintenance.

All deployment-time settings are env-driven via `PMOS_*` variables (parsed through the `pydantic-settings` library with optional `.env` support), including the API bind host and port, CORS origins, predictor identifier and model path, the active ingestor and its mode-specific options (replay path, serial port and baud rate), and the default RPM used for stream predictions. This configuration story is what allows the same code to switch from a laptop demo running synthetic data to a production deployment reading a live UART feed without any source modification.

7.8 Extensibility Points

The deployment exposes two formal extension points, both designed around abstract base classes so that adding a new component does not require modifying any consumer code.

Replacing the model. A new architecture is integrated by implementing the `Predictor ABC` (defined in `pmos.models.base`), registering the class in `pmos/models/registry.py`, and setting `PMOS_PREDICTOR_ID` and `PMOS_PREDICTOR_PATH` at deployment. The `Prediction` schema, the `/api/meta` endpoint, and both batch and stream endpoints continue to work unchanged; only the model card and the README require an update.

Adding a data source. A new ingestor is integrated by subclassing `BaseIngestor` (defined in `pmos.streaming.ingestor`), implementing its asynchronous `events()` iterator, adding a branch to `build_ingestor` in `pmos.api.deps`, and selecting it via `PMOS_INGESTOR`. This is how the system can be extended from the three shipped ingestors to, for example, an MQTT broker or an OPC-UA endpoint in a production facility.

Together with the ISO 20816-3 mapping, these extension points position the `Maintenance_OS` deployment not as a single fixed pipeline but as a small framework: a versioned model artefact, a swappable data source, a standards-compliant severity vocabulary, and an operator-facing UI designed for at-a-glance industrial use.

8. Conclusion and Future Work

This final chapter concludes the project report by summarizing the key contributions of our work, candidly discussing the experimental and logistical limitations encountered, and outlining the next steps for advancing vibration-based fault diagnostics. The transition from fault classification to severity estimation represents a significant step toward making predictive maintenance more actionable for industrial applications.

8.1 Summary of Contributions

This project successfully developed an end-to-end framework for identifying mechanical faults and quantifying their severity levels in rotating machinery. Our work extended the baseline established by previous researchers by introducing a multi-level severity estimation layer for unbalance, misalignment, and mechanical looseness. Key technical milestones achieved during the project include:

- **Physical Rig Optimization:** Seven iterative setup modifications to eliminate baseline mechanical noise and resonance. This process resolved a critical initial failure where the system could not distinguish between a “Normal” state and an “Unbalanced” state.
- **High-Integrity Data Ingestion:** A strict synchronization protocol that reduced 73,000 raw frames to 21,304 high-quality triaxial samples, ensuring that every digital snapshot used for training was a physically accurate representation of the machine’s state.
- **MiniRocket Implementation:** After evaluating multiple feature-based and deep learning architectures, we identified the MiniRocket + Ridge Classifier pipeline as the superior solution, achieving near-perfect precision and recall scores across all severity levels.
- **Industry-Ready Deployment:** A successful transition from a high-performance training environment to a lightweight, local deployment stack featuring a modular FastAPI backend (REST and WebSocket), a pluggable ingestion layer with synthetic, replay, and live serial sources, and a custom vanilla-JS frontend with ISO 20816-3 severity mapping.

8.2 Experimental and Practical Limitations

Despite the high performance of the final system, several limitations influenced the project’s development and scope.

8.2.1 Supply Chain and Logistics

The original scope of the project included severity estimation for bearing faults. However, the simulation of varied bearing fault intensities required the acquisition of multiple additional bearings with specific defects. Due to supply-chain issues and logistical constraints, these components could not be obtained within the project’s timeline. Consequently, while bearing-fault *classification* was achieved, severity *quantification* for this specific category remains for future study.

8.2.2 Interpretability of the Selected Model

While the MiniRocket architecture provided the highest predictive performance, it presents a challenge regarding physical interpretability. Unlike the manual feature-engineering path, where specific metrics like RMS or dominant frequency can be directly mapped to machine physics, the 10,000 random convolutional kernels used by MiniRocket act as a “black box”. For industrial users who require a deep understanding of *why* a specific severity level was triggered, this lack of direct transparency is a notable trade-off for high accuracy.

8.3 Future Work

The findings of this project provide a solid foundation for several promising research directions.

8.3.1 Progressive Severity and Trajectory Modeling

One of the most valuable extensions of this work would be the development of progressive fault tracking. While our current system classifies faults into discrete “Low,” “Medium,” and “High” buckets, future iterations should aim to predict the *trajectory* of fault progression. This would allow maintenance teams to estimate exactly when a “Medium” severity fault will transition to a “High” severity state, enabling more precise repair scheduling.

8.3.2 Broadening the Modeling Search

Due to time and compute constraints, we prioritized the stability of the MiniRocket default configuration. Future work should involve a more exhaustive search of modeling techniques, including deeper CNN architectures, Vision Transformers for vibration imaging, and hybrid physics-informed neural networks. If provided with extended testing time and greater compute resources, these alternative techniques could potentially match MiniRocket’s accuracy while offering improved interpretability or smaller memory footprints.

8.4 Concluding Remarks

This work demonstrates that the successful application of AI to industrial machinery requires a careful balance of mechanical engineering and data science. By prioritizing physical rig optimization before digital modeling, our team was able to build a system that is not only accurate but robust enough for local deployment. The resulting severity-estimation framework provides a practical tool for modern predictive maintenance, moving us closer to a future where machine failures are not just predicted but fully *quantified*.

Bibliography

- [1] ABB. (2023). *ABB survey reveals unplanned downtime costs \$125,000 per hour.*
- [2] Chen, M. et al. (2025). *Self-supervised learning for machinery fault diagnosis.*
- [3] Chen, T. and Guestrin, C. (2016). *XGBoost: A Scalable Tree Boosting System.*
- [4] Dempster, A. et al. (2021). *MiniROCKET: A Very Fast (Almost) Deterministic Transform for Time Series Classification.*
- [5] FastAPI. (2024). *FastAPI framework, high performance, easy to learn, fast to code, ready for production.*
- [6] Four Jaw Manufacturing Analytics. (2025). *The cost of downtime in manufacturing.*
- [7] Institute for Supply Management. (2024). *The monthly metric: Unscheduled downtime.*
- [8] Ismail Fawaz, H. et al. (2019). *Deep Learning for Time Series Classification: A Review.*
- [9] Ismail Fawaz, H. et al. (2020). *InceptionTime: Finding AlexNet for Time Series Classification.*
- [10] StrainLabs. (2025). *Planned vs. unplanned downtime in manufacturing.*
- [11] Zhang, Y. et al. (2023). *A Contrastive Learning-Based Fault Diagnosis Method for Rotating Machinery.*
- [12] Zhao, K. et al. (2025). *Enhanced prognostic reliability for rotating machinery using neural networks.*